

WELCOME

CCBP 4.0

21st Century Skills

Lists

Recap

Compound Assignment

```
a = a + 1
```



```
a += 1
```

Compound Assignment Operator

Code

```
a = 10
a += 1
print(a)
```

Output

```
11
```

Recap

Floor Division

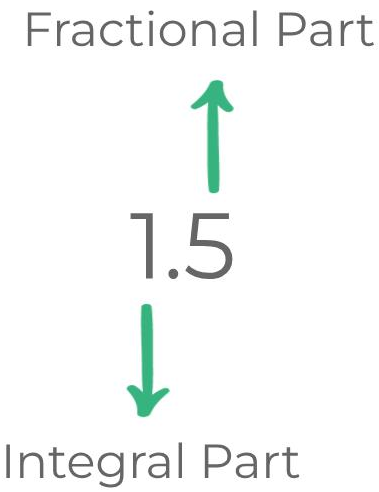
To find the integral part of quotient, we use Floor Division Operator(//)

Code

```
print(3 // 2)
```

Output

```
1
```



Recap

Single and Double Quotes

Code

```
is_same = ('Cricket' == "Cricket")
print(is_same)
```

Output

```
True
```

Recap

Escape Characters

Escape Characters start with a backslash in Python

`\n`	New Line	`\'`	Single Quote
`\t`	Tab Space	`\"`	Double Quote
`\\`	Backslash		and more...

Recap

Printing String with Quotes

Code

```
print('It\'s Python')
```

Output

```
It's Python
```

The **backslash ** character here tells Python not to consider the next character as the ending of the string

Daily Challenge

Identify The Mistake

Code

```
n = int(input())
for i in range(1, n+1):
    spaces = " "*(n - i)
    left_nums = ""
    right_nums = ""
    for j in range(1, i+1):
        left_nums = str(j) + left_nums
        right_nums =  right_nums + str(j)
    print(spaces + left_nums + right_nums)
```

Input

```
3
```

Expected Output

```
1
212
32123
```

NXT WAVE

Daily Challenge

Identify The Mistake

Code

```
n = int(input())
for i in range(1, n+1):
    spaces = " "*(n - i)
    left_nums = ""
    right_nums = ""
    for j in range(1, i+1):
        left_nums = str(j) + left_nums
        right_nums =  right_nums + str(j)
    print(spaces + left_nums + right_nums[1:])
```

Input

```
3
```

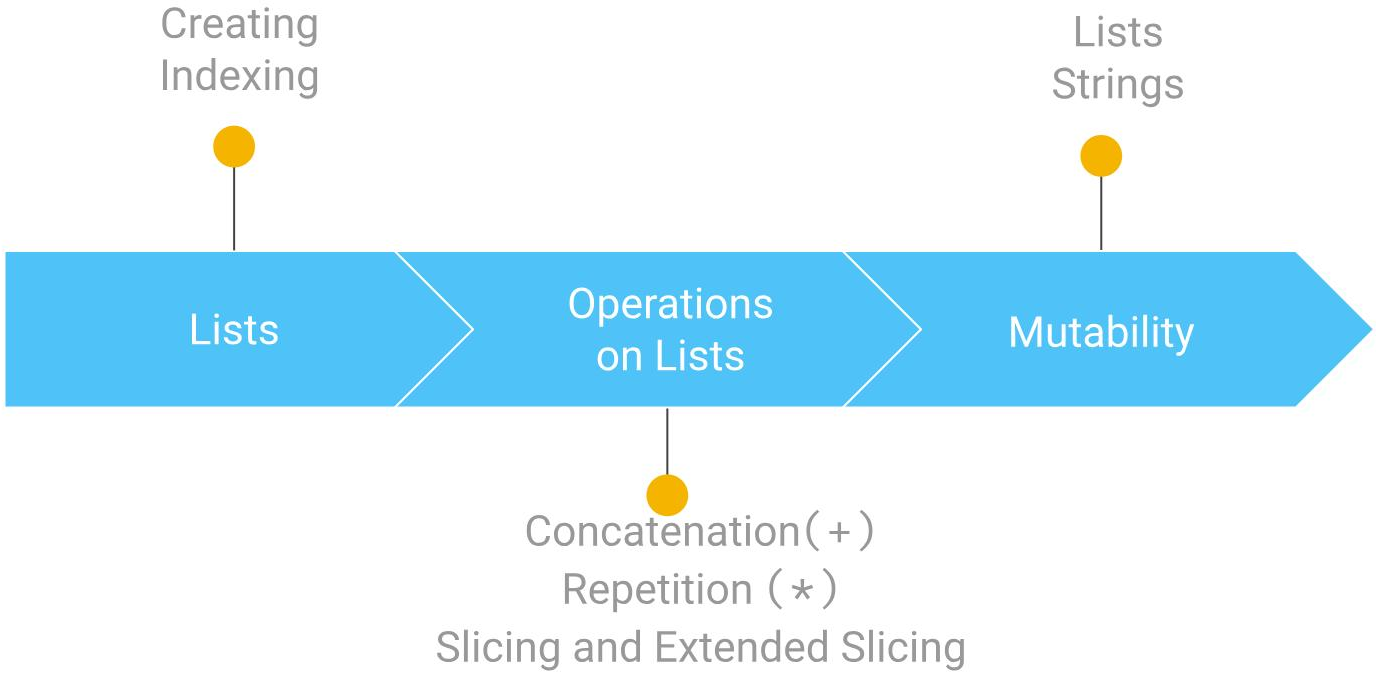
Output

```
1
212
32123
```

NXT WAVE

Agenda for Today's Session

Lists



Introduction

Data Structures

Data Structures allow us to
store and organize data efficiently

This will allow us to easily access and
perform operations on the data



Data Structures

In Python, there are four built-in data structures

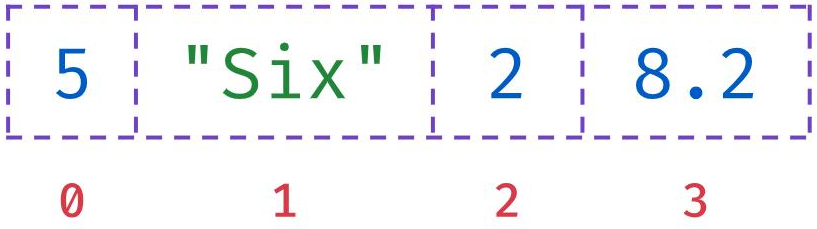
- `List`
- `Tuple`
- `Set`
- `Dictionary`

Data Structures

List

List is the most versatile Python data structure

Holds an ordered sequence of items



Creating a List

Created by enclosing elements within [square] brackets

Each item is separated by a **comma**

Code

```
a = 2
list_a = [5, "Six", a, 8.2]
```

Creating a List

Code

```
a = 2
list_a = [5, "Six", a, 8.2]
print(type(list_a))
print(list_a)
```

Output

```
<class 'list'>
[5, 'Six', 2, 8.2]
```

Creating a List of Lists

Code

```
a = 2
list_a = [5, "Six", a, 8.2]
list_b = [1, list_a]
print(list_b)
```

Output

```
[1, [5, 'Six', 2, 8.2]]
```

Length of List

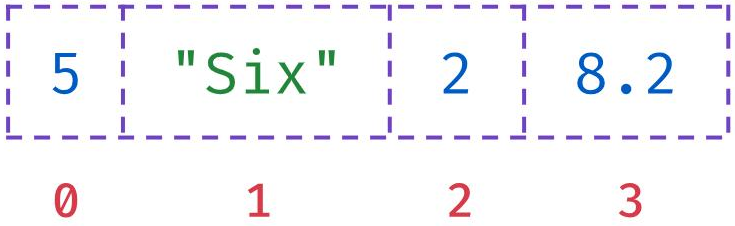
Code

```
a = 2
list_a = [5, "Six", a, 8.2]
print(len(list_a))
```

Output

```
4
```

Accessing List Elements



To access elements of a list, we use **Indexing**

Code

```
a = 2
list_a = [5, "Six", a, 8.2]
print(list_a[1])
```

Output

```
Six
```

Iterating Over a List

Code

```
a = 2
list_a = [5, "Six", a, 8.2]
for item in list_a:
    print(item)
```

Output

```
5
Six
2
8.2
```

Operations on List

Concatenation

Code

```
list_a = [1, 2, 3]
list_b = ["a", "b", "c"]
list_c = list_a + list_b
print(list_c)
```

Output

```
[1, 2, 3, 'a', 'b', 'c']
```

Similar to strings,

+ **operator** concatenates lists

NXT WAVE

Adding Items to List

Code

```
list_a = []
print(list_a)
for i in range(1,4):
    list_a += [i]
print(list_a)
```



Initializes empty list

Operations on List

Adding Items to List

Code

```
list_a = []
print(list_a)
for i in range(1,4):
    list_a += [i]
print(list_a)
```

Adding Items to List

Code

```
list_a = []
print(list_a)
for i in range(1,4):
    list_a += [i]
print(list_a)
```

Output

```
[]
[1, 2, 3]
```

Operations on List

Repetition

Code

```
list_a = [1, 2]
list_b = list_a * 3
print(list_b)
```

* Operator repeats lists

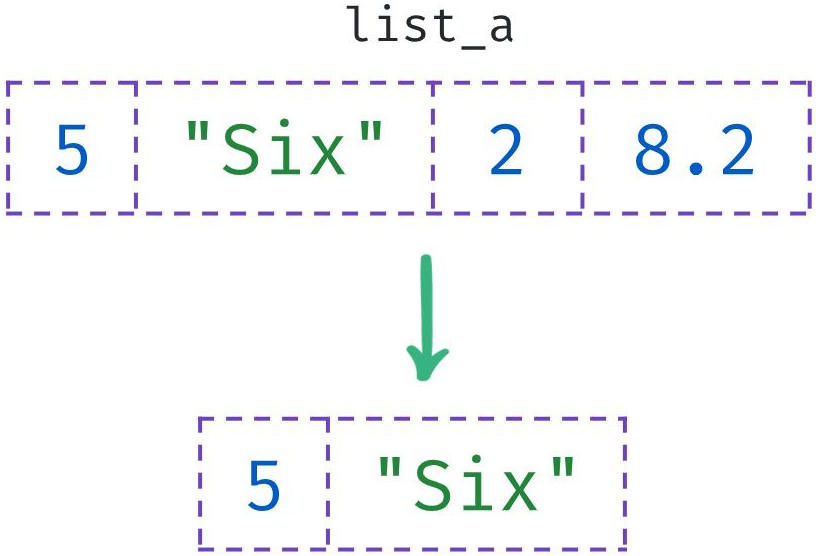
Output

```
[1, 2, 1, 2, 1, 2]
```

Obtain Part of a List

Obtaining a part of a list is called

List Slicing



Obtain Part of a List

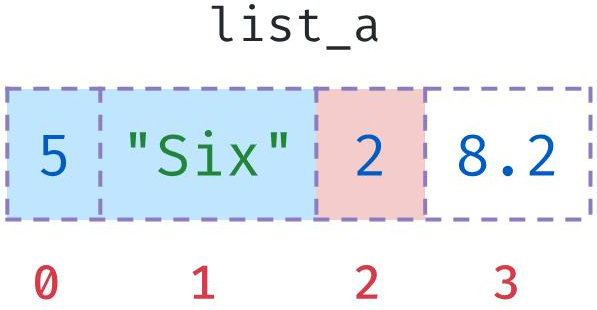
Slicing

Code

```
list_a = [5, "Six", 2, 8.2]
list_b = list_a[:2]
print(list_b)
```

Output

```
[5, 'Six']
```

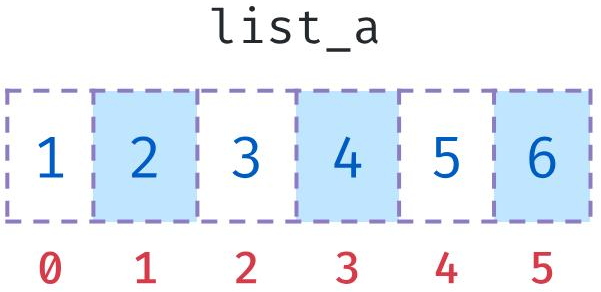


Extended Slicing

Similar to string extended slicing,

we can extract

alternate items using **step**



Extended Slicing

Code

```
list_a = ["R", "B", "G", "O", "W"]
list_b = list_a[0:5:3]
print(list_b)
```

Output

```
['R', 'O']
```

Converting to List

`list(sequence)`

Takes a sequence and converts it into list

Code

```
color = "Red"
list_a = list(color)
print(list_a)
```

Converting to List

Code

```
color = "Red"
list_a = list(color)
print(list_a)
```

Output

```
['R', 'e', 'd']
```

Converting to List

Code

```
list_a = list(range(4))
print(list_a)
```

Converting to List

Code

```
list_a = list(range(4))
print(list_a)
```

Output

```
[0, 1, 2, 3]
```

Lists are Mutable

Lists can be modified

Items at any position can be updated

Code

```
list_a = [1, 2, 3, 5]
print(list_a)
list_a[3] = 4
print(list_a)
```

Lists are Mutable

Code

```
list_a = [1, 2, 3, 5]
print(list_a)
list_a[3] = 4
print(list_a)
```

Output

```
[1, 2, 3, 5]
[1, 2, 3, 4]
```

Strings

Changing a Character

Code

```
message = "sea you soon"
message[2] = "e"
print(message)
```



Can we modify a string like this?

Changing a Character

Code

```
message = "sea you soon"
message[2] = "e"
print(message)
```

Output

```
TypeError:
'str' object does not
support item assignment
```

Strings are Immutable

(Can't be modified)

Key Takeaways

- Creating a List
- Accessing List Items
- List Concatenation
- List Repetition



Key Takeaways

- Slicing and Extended Slicing
- Converting to Lists
- List and String Mutability

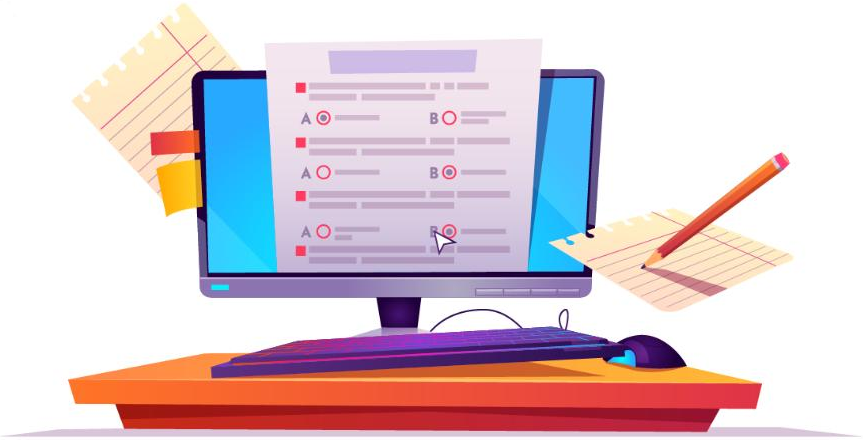


Next Session

Lists

(To be Continued ...)

Practice



Feedback



Daily Challenge

Identify The Mistake

Code

```
animals = ['cat', 'dog']
wild_animals = ['Tiger', 'Fox']
animals += wild_animals
print(animals)
```

Expected Output

```
['cat', 'dog', ['Tiger', 'Fox']]
```

NXT WAVE

NXT WAVE

THANK YOU



ALL THE BEST